

2.To perform basic NumPy array operations using synthetic data.

In [1]: *#1. Create a NumPy Array and Display It*

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("Array:", arr)
```

Array: [10 20 30 40 50]

In [2]: *#2. Find Sum and Mean of Array Elements*

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("Sum =", np.sum(arr))
print("Mean =", np.mean(arr))
```

Sum = 150

Mean = 30.0

In [3]: *#3. Perform Addition on Array Elements*

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
print("Original Array:", arr)
print("After Adding 10:", arr + 10)
```

Original Array: [1 2 3 4 5]

After Adding 10: [11 12 13 14 15]

In [4]: *#4. Find Maximum and Minimum Values*

```
import numpy as np
arr = np.array([15, 25, 5, 40, 30])
print("Maximum =", np.max(arr))
print("Minimum =", np.min(arr))
```

Maximum = 40

Minimum = 5

In [5]: *#5. Demonstrate Indexing and Slicing*

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("First Element:", arr[0])
print("Slice (1 to 3):", arr[1:4])
```

First Element: 10

Slice (1 to 3): [20 30 40]

In [6]: #6. Program

```
import numpy as np
# Generate synthetic data
data = np.array([10, 20, 30, 40, 50])
print("Original Array:")
print(data)
# Arithmetic Operations
print("\nAddition (+5):")
print(data + 5)
print("\nMultiplication (*2):")
print(data * 2)
# Statistical Operations
print("\nMean:", np.mean(data))
print("Sum:", np.sum(data))
print("Maximum:", np.max(data))

print("Minimum:", np.min(data))
# Indexing
print("\nFirst Element:", data[0])
# Slicing
print("Elements from index 1 to 3:")
print(data[1:4])
# Reshaping
matrix = data.reshape(5, 1)
print("\nReshaped Array:")
print(matrix)
```

Original Array:
[10 20 30 40 50]

Addition (+5):
[15 25 35 45 55]

Multiplication (*2):
[20 40 60 80 100]

Mean: 30.0
Sum: 150
Maximum: 50
Minimum: 10

First Element: 10
Elements from index 1 to 3:
[20 30 40]

Reshaped Array:
[[10]
 [20]
 [30]
 [40]
 [50]]

In [7]: #1. Load the CSV File Using NumPy

```
import pandas as pd
import numpy as np

df = pd.read_csv(r"C:\Users\Smit\Downloads\train.csv")

data = df.to_numpy()

print("Shape =", data.shape)
print(data[:5])
```

Shape = (1460, 81)

```
[[1 60 'RL' 65.0 8450 'Pave' nan 'Reg' 'Lvl' 'AllPub' 'Inside' 'Gtl'
  'CollgCr' 'Norm' 'Norm' '1Fam' '2Story' 7 5 2003 2003 'Gable' 'CompShg'
  'VinylSd' 'VinylSd' 'BrkFace' 196.0 'Gd' 'TA' 'PConc' 'Gd' 'TA' 'No'
  'GLQ' 706 'Unf' 0 150 856 'GasA' 'Ex' 'Y' 'SBrkr' 856 854 0 1710 1 0 2
  1 3 1 'Gd' 8 'Typ' 0 nan 'Attchd' 2003.0 'RFn' 2 548 'TA' 'TA' 'Y' 0 61
  0 0 0 0 nan nan nan 0 2 2008 'WD' 'Normal' 208500]
[2 20 'RL' 80.0 9600 'Pave' nan 'Reg' 'Lvl' 'AllPub' 'FR2' 'Gtl'
  'Veenker' 'Feedr' 'Norm' '1Fam' '1Story' 6 8 1976 1976 'Gable'
  'CompShg' 'MetalSd' 'MetalSd' 'None' 0.0 'TA' 'TA' 'CBlock' 'Gd' 'TA'
  'Gd' 'ALQ' 978 'Unf' 0 284 1262 'GasA' 'Ex' 'Y' 'SBrkr' 1262 0 0 1262 0
  1 2 0 3 1 'TA' 6 'Typ' 1 'TA' 'Attchd' 1976.0 'RFn' 2 460 'TA' 'TA' 'Y'
  298 0 0 0 0 0 nan nan nan 0 5 2007 'WD' 'Normal' 181500]
[3 60 'RL' 68.0 11250 'Pave' nan 'IR1' 'Lvl' 'AllPub' 'Inside' 'Gtl'
  'CollgCr' 'Norm' 'Norm' '1Fam' '2Story' 7 5 2001 2002 'Gable' 'CompShg'
  'VinylSd' 'VinylSd' 'BrkFace' 162.0 'Gd' 'TA' 'PConc' 'Gd' 'TA' 'Mn'
  'GLQ' 486 'Unf' 0 434 920 'GasA' 'Ex' 'Y' 'SBrkr' 920 866 0 1786 1 0 2
  1 3 1 'Gd' 6 'Typ' 1 'TA' 'Attchd' 2001.0 'RFn' 2 608 'TA' 'TA' 'Y' 0
  42 0 0 0 0 nan nan nan 0 9 2008 'WD' 'Normal' 223500]
[4 70 'RL' 60.0 9550 'Pave' nan 'IR1' 'Lvl' 'AllPub' 'Corner' 'Gtl'
  'Crawfor' 'Norm' 'Norm' '1Fam' '2Story' 7 5 1915 1970 'Gable' 'CompShg'
  'Wd Sdng' 'Wd Shng' 'None' 0.0 'TA' 'TA' 'BrkTil' 'TA' 'Gd' 'No' 'ALQ'
  216 'Unf' 0 540 756 'GasA' 'Gd' 'Y' 'SBrkr' 961 756 0 1717 1 0 1 0 3 1
  'Gd' 7 'Typ' 1 'Gd' 'Detchd' 1998.0 'Unf' 3 642 'TA' 'TA' 'Y' 0 35 272
  0 0 0 nan nan nan 0 2 2006 'WD' 'Abnorml' 140000]
[5 60 'RL' 84.0 14260 'Pave' nan 'IR1' 'Lvl' 'AllPub' 'FR2' 'Gtl'
  'NoRidge' 'Norm' 'Norm' '1Fam' '2Story' 8 5 2000 2000 'Gable' 'CompShg'
  'VinylSd' 'VinylSd' 'BrkFace' 350.0 'Gd' 'TA' 'PConc' 'Gd' 'TA' 'Av'
  'GLQ' 655 'Unf' 0 490 1145 'GasA' 'Ex' 'Y' 'SBrkr' 1145 1053 0 2198 1 0
  2 1 4 1 'Gd' 9 'Typ' 1 'TA' 'Attchd' 2000.0 'RFn' 3 836 'TA' 'TA' 'Y'
  192 84 0 0 0 0 nan nan nan 0 12 2008 'WD' 'Normal' 250000]]
```

In [8]: #2. Find the Average Sale Price

```
import pandas as pd
import numpy as np

df = pd.read_csv(r"C:\Users\Smit\Downloads\train.csv")

data = df.to_numpy()

sale_price = data[:, 80].astype(float)

print("Average Sale Price =", np.mean(sale_price))
```

Average Sale Price = 180921.19589041095

In [9]: *#3. Find Maximum and Minimum Sale Price*

```
import pandas as pd
import numpy as np

df = pd.read_csv(r"C:\Users\Smit\Downloads\train.csv")

data = df.to_numpy()

sale_price = data[:, 80].astype(float)

print("Maximum Sale Price =", np.max(sale_price))
print("Minimum Sale Price =", np.min(sale_price))
```

```
Maximum Sale Price = 755000.0
Minimum Sale Price = 34900.0
```

In [10]: *#4. Count Houses by Building Type*

```
import pandas as pd
import numpy as np

df = pd.read_csv(r"C:\Users\Smit\Downloads\train.csv")

data = df.to_numpy()

bldg = data[:, 15]

print("1Fam =", np.sum(bldg == "1Fam"))
print("2fmCon =", np.sum(bldg == "2fmCon"))
print("Duplex =", np.sum(bldg == "Duplex"))
print("Twnhs =", np.sum(bldg == "Twnhs"))
print("TwnhsE =", np.sum(bldg == "TwnhsE"))
```

```
1Fam = 1220
2fmCon = 31
Duplex = 52
Twnhs = 43
TwnhsE = 114
```

In [11]: *#5. Calculate Average Sale Price by Overall Quality*

```
import pandas as pd
import numpy as np

df = pd.read_csv(r"C:\Users\Smit\Downloads\train.csv")

data = df.to_numpy()

quality = data[:, 17].astype(int)
sale_price = data[:, 80].astype(float)

for q in range(1, 11):
    avg = np.mean(sale_price[quality == q])
    print("Quality", q, "Average Sale Price =", avg)
```

```
Quality 1 Average Sale Price = 50150.0
Quality 2 Average Sale Price = 51770.333333333336
Quality 3 Average Sale Price = 87473.75
Quality 4 Average Sale Price = 108420.6551724138
Quality 5 Average Sale Price = 133523.34760705288
Quality 6 Average Sale Price = 161603.0347593583
Quality 7 Average Sale Price = 207716.42319749217
Quality 8 Average Sale Price = 274735.53571428574
Quality 9 Average Sale Price = 367513.0232558139
Quality 10 Average Sale Price = 438588.3888888889
```

In []: